

PROJECT PROPOSAL

Team Information

Project Title: **GIT it!**

Project Short Description: ***GIT it!* is a web-based, scenario-driven learning platform with fading instructional scaffolding that helps students practice Git through realistic repository failure scenarios, live DAG visualization, and state-based evaluation—building transferable skills in a consequence-safe environment.**

Team Code: **2526-sem2-it332-34**

PART 1: Introduction

Version control systems have become essential tools in modern software development, enabling teams to track code changes, coordinate collaborative work, and maintain the integrity of software projects. Among these tools, Git has emerged as the industry standard and is widely adopted in professional programming environments (Zagalsky et al., 2015). Despite its importance, a persistent gap exists between how Git is taught in undergraduate programs and the competencies graduates are expected to demonstrate in the workforce. Garousi et al. (2019) identified version control as one of the most consistently cited unmet competencies among computing graduates internationally, and this finding is reinforced by Garousi et al. (2020), who documented that the practical version control deficiencies of software engineering graduates lead employers to invest heavily in post-hire training — with consequences ranging from elevated onboarding costs to disqualification at the technical screening stage of hiring. Akdur (2022) confirmed that version control ranks among the top deficiencies flagged by industry managers evaluating new hires, a pattern that has remained stable across multiple geographic and institutional contexts.

Research consistently shows that novice developers lack a clear mental model of Git operations such as branching, merging, and conflict resolution (Isomöttönen & Cochez, 2014). Milliken et al. (2021) confirmed that this gap persists across skill levels — command familiarity does not equal command competence, and users at all experience levels routinely fail to understand how repository state changes in response to the commands they execute. Without this foundational mental model, students memorize commands without grasping how repository states change during collaboration. Embury (2019) documented that students exhibit poor Git practices and persistent misconceptions even after Git is formally introduced in coursework, and Beckman et al. (2020) found that students struggle to understand version control concepts even when Git and GitHub are integrated directly into university curricula. When conflicts arise or branches diverge, students resort to destructive behaviors: deleting and re-cloning repositories, overwriting teammates' work, or delegating all version control to a single group member (Milliken et al., 2021; Gitinabard et al., 2020). Left unaddressed, these behaviors carry directly into professional practice — graduates enter development teams unable to manage branching strategies, resolve conflicts independently, or recover from common repository errors (Guerrero-Higuera et al., 2020; Garousi et al., 2020). Wagner and Thurner (2024) argued that the field must move beyond command memorization toward conceptual understanding and practical workflows, identifying the absence of pedagogically grounded Git learning environments as a key barrier to achieving this shift.

These patterns are equally evident at Cebu Institute of Technology – University (CIT-U). A survey of forty-four third-year BSIT students enrolled in Information Management 2 and Applications Development revealed a widespread cycle of avoidance, dependency, and destructive workarounds: 91% reported pasting Git errors directly into ChatGPT or Gemini and executing whatever command was returned without understanding it — yet only 23% had ever consulted the

official Git documentation; 75% described teams that simply "picked one side" and discarded a teammate's code when merge conflicts arose; 45% deleted the local folder and re-cloned the repository to escape a conflict; 16% abandoned the repository entirely and created a new one to resolve a conflict — representing the most extreme avoidance behavior documented in the survey. These patterns align with broader evidence that digital learning adoption barriers are particularly pronounced in Philippine higher education contexts, where locally contextualized instructional tools remain scarce (Illescas et al., 2023). When asked what they most wished a Git learning platform would teach, student responses converged on three core needs: safely resolving merge conflicts, undoing accidental commits or pushes, and practicing these scenarios step-by-step in a simulated environment before working on real projects.

The problem addressed by this study is the inability of third-year BSIT students at CIT-U to apply Git version control competently under realistic collaborative conditions — evidenced by pervasive AI dependency without comprehension (89%), destructive conflict resolution habits (71%), and avoidance behaviors ranging from re-cloning to abandoning repositories altogether. This problem is compounded by the complete absence of any locally available instructional tool that provides consequence-safe, scenario-driven practice with state-based evaluation, visual repository feedback, and difficulty-differentiated instructional support tailored to this population. The resulting skills gap has direct professional consequences: graduates who cannot independently manage branching strategies, resolve merge conflicts, or recover from destructive repository states are routinely screened out at the technical evaluation stage of software industry hiring or require extended onboarding at significant cost to their employers (Garousi et al., 2019; Garousi et al., 2020; Akdur, 2022).

Although tools such as Learn Git Branching and GitHub Classroom exist, none combine realistic repository failure states with: (1) a structured orientation unit that builds the foundational Git mental model students need before entering scenario-driven practice; (2) difficulty-differentiated instructional support that progressively reduces scaffolding as students engage with harder difficulty levels; (3) a live Directed Acyclic Graph (DAG) visualization that updates in real time after every simulator-processed command; (4) text-based repository consequence feedback that summarizes how the student's command changed the simulated repository state without labeling it correct or incorrect; (5) command-limit enforcement that discourages brute-force command entry and promotes deliberate reasoning; (6) adaptive scenario variation that serves structurally different repository templates on retry to promote genuine skill transfer; and (7) a lesson-centered content structure in which each learning unit contains lessons, and scenario practice is accessed through the lesson that teaches the relevant Git concept. GIT it! addresses this gap through a scenario-driven learning platform delivered as a browser-accessible web application.

PART 2: Objectives

Module 1 — Local Repository Foundations

General Objective

Develop students' ability to independently manage local Git repositories during the system evaluation period through repeated scenario-based practice at a Scenario Completion Rate of $\geq 80\%$, including repository initialization, cloning, staging and committing, partial staging, amending commits, and unstaging and discarding changes.

Competency achievement for this module shall be evaluated using Command Accuracy Rate (CAR), Hard-Level Completion Rate (HLCR), and Average Retry Count (ARC) across active Local Repository Foundations scenario sessions.

Specific Objectives

SO 1.1 — Initializing Repositories

Improve Command Accuracy for initializing local Git repositories to $\geq 70\%$ across varied project structures and repository contexts by the end of the deployment and system testing phase.

SO 1.2 — Cloning Remote Repositories

Improve Command Accuracy for cloning remote repositories to $\geq 70\%$ across varied URL formats, HTTPS vs SSH contexts, and destination directory conditions by the end of the deployment and system testing phase.

SO 1.3 — Staging and Committing

Improve Command Accuracy for staging file changes and creating valid commits with descriptive, imperative-mood messages to $\geq 70\%$ across varied file-selection patterns — including single files, directories, and partial staging by the end of the deployment and system testing phase.

SO 1.4 — Partial Staging

Improve Command Accuracy for performing partial staging to $\geq 70\%$ across varied hunk-selection contexts by the end of the deployment and system testing phase.

SO 1.5 — Amending Commits

Improve Command Accuracy for amending the most recent commit — including message correction and staged-content modification — to $\geq 70\%$ across varied amendment contexts and repository states by the end of the deployment and system testing phase.

SO 1.6 — Unstaging and Discarding Changes

Improve Command Accuracy for unstaging files and discarding working-tree changes to $\geq 70\%$ across varied staging contexts by the end of the deployment and system testing phase.

SO 1.7 — Independent Local Repository Management

Achieve Hard-Level Completion Rate within Module 1 of $\geq 70\%$ across multi-step scenarios requiring independent management of a local Git repository — including initialization, cloning, staging, committing, amending, and unstaging — without scaffolding support by the end of the acceptance testing phase.

SO 1.8 — Efficient Repository-State Reasoning

Reduce Average Retry Count across all Module 1 scenarios to ≤ 2 across all difficulty levels within the module by the end of the deployment and system testing phase, indicating students are applying deliberate repository-state reasoning rather than repeated trial-and-error command entry.

Module 2 — Branching and Collaboration

General Objective

Develop students' ability to create, manage, and merge Git branches, coordinate remote collaboration workflows through push, fetch, pull, and stash operations, reconcile diverged histories, and recover remote branches during the system evaluation period through repeated scenario-based practice at a Scenario Completion Rate of $\geq 80\%$.

Competency achievement for this module shall be evaluated using Command Accuracy Rate (CAR), Hard-Level Completion Rate (HLCR), and Average Retry Count (ARC) across active Branching and Collaboration scenario sessions.

Specific Objectives

SO 2.1 — Creating and Switching Branches

Improve Command Accuracy for creating and switching branches to $\geq 70\%$ across varied repository topologies, and base branch conditions by the end of the deployment and system testing phase.

SO 2.2 — Branch Naming Conventions and Housekeeping

Improve Command Accuracy for applying valid branch naming conventions and managing stale local branches to $\geq 70\%$ across varied workflow descriptions and project contexts by the end of the deployment and system testing phase.

SO 2.3 — Stashing Work in Progress

Improve Command Accuracy for stashing and restoring work in progress to $\geq 70\%$ across varied mid-task context-switch scenarios by the end of the deployment and system testing phase.

SO 2.4 — Pushing to a Remote

Improve Command Accuracy for pushing local branches to a remote repository — including setting upstream tracking and handling rejected pushes to $\geq 70\%$ across varied remote-state conditions and branch configurations by the end of the deployment and system testing phase.

SO 2.5 — Fetching and Pulling from a Remote

Improve Command Accuracy for distinguishing between fetch-only and fetch-with-integrate workflows to $\geq 70\%$ across varied remote-state conditions by the end of the deployment and system testing phase.

SO 2.6 — Reconciling Diverged Local and Remote Histories

Improve Command Accuracy for reconciling local and remote histories before a successful push to $\geq 70\%$ across varied local-remote divergence conditions by the end of the deployment and system testing phase.

SO 2.7 — Completing Branch Merges

Improve Command Accuracy for completing branch merges — including fast-forward and three-way merges — to $\geq 70\%$ across varied branch states and divergence depths by the end of the deployment and system testing phase.

SO 2.8 — Squash Merging

Improve Command Accuracy for applying squash merges to consolidate feature-branch history to $\geq 70\%$ across varied branch sizes and commit densities by the end of the deployment and system testing phase.

SO 2.9 — Deleting and Recovering Remote Branches

Improve Command Accuracy for deleting stale remote branches and recovering accidentally deleted remote branches to $\geq 70\%$ across varied merged and unmerged remote-branch conditions by the end of the deployment and system testing phase.

SO 2.10 — Independent Branch and Collaboration Management

Achieve Hard-Level Completion Rate within Module 2 of $\geq 70\%$ across multi-step scenarios requiring independent branch creation, remote push and pull, diverged-history reconciliation, merge completion, and remote branch recovery without scaffolding support by the end of the acceptance testing phase.

SO 2.11 — Reduced Trial-and-Error Branching

Reduce Average Retry Count across all Module 2 scenarios to ≤ 2 across all difficulty levels within the module by the end of the deployment and system testing phase, demonstrating that students are applying consequence-aware branching reasoning rather than command-pattern repetition.

Module 3 — Conflict Resolution

General Objective

Develop students' ability to resolve conflicts manually and with merge tools, and selectively apply commits through cherry-picking during the system evaluation period through repeated scenario-based practice at a Scenario Completion Rate of $\geq 80\%$.

Competency achievement for this module shall be evaluated using Command Accuracy Rate (CAR), Hard-Level Completion Rate (HLCR), Retry Transfer Accuracy (RTA), and Average Retry Count (ARC) across active Conflict Resolution scenario sessions.

Specific Objectives

SO 3.1 — Resolving Merge Conflicts Manually

Improve Command Accuracy for manually resolving merge conflicts — correctly identifying conflict markers, editing affected files, and completing the merge — to $\geq 70\%$ across varied conflict patterns and file types by the end of the deployment and system testing phase.

SO 3.2 — Resolving Conflicts Using a Merge Tool

Improve Command Accuracy for configuring a merge tool (git config merge.tool) and completing a conflict resolution workflow to $\geq 70\%$ across varied conflict complexities and merge tool contexts by the end of the deployment and system testing phase.

SO 3.3 — Cherry-Picking Commits

Improve Command Accuracy for selectively applying commits from one branch to another using git cherry-pick to $\geq 70\%$ across varied commit-selection contexts — including single commits, no-commit mode, and abort scenarios by the end of the deployment and system testing phase.

SO 3.4 — Independent Conflict Resolution

Achieve Hard-Level Completion Rate within Module 3 of $\geq 70\%$ across multi-step scenarios requiring independent manual conflict resolution, mergetool workflow completion, and cherry-pick operations without scaffolding support by the end of the acceptance testing phase.

SO 3.5 — Transferable Conflict-Resolution Reasoning

Achieve Retry Transfer Accuracy of $\geq 65\%$ across structurally changed retry variant sessions and reduce Average Retry Count to ≤ 2 across all Module 3 scenarios by the end of the

deployment and system testing phase, demonstrating that students apply Git conflict-resolution concepts to changed repository topologies rather than reproducing memorized command sequences.

Module 4 — Advanced Recovery and History

General Objective

Develop students' ability to safely reverse pushed commits, recover lost work following destructive operations, complete interactive rebase sequences, and execute full multi-step rebase recovery operations during the system evaluation period through repeated scenario-based practice at a Scenario Completion Rate of $\geq 80\%$.

Competency achievement for this module shall be evaluated using Command Accuracy Rate (CAR), Hard-Level Completion Rate (HLCR), Average Retry Count (ARC), and Retry Transfer Accuracy (RTA) across active Advanced Recovery and History scenario sessions.

Specific Objectives

SO 4.1 — Recovering from Hard Resets

Improve Command Accuracy for restoring lost work following a destructive command to $\geq 70\%$ across varied reset depths and repository states by the end of the deployment and system testing phase.

SO 4.2 — Reversing Pushed Commits Safely

Improve Command Accuracy for safely reversing already-pushed commits to $\geq 70\%$ across varied commit-history depths and push contexts — distinguishing revert from reset in scenarios where shared history must be preserved by the end of the deployment and system testing phase.

SO 4.3 — Completing Rebase Recovery Sequences

Improve Command Accuracy for completing full rebase recovery sequences — without dropping, duplicating, or orphaning commits — to $\geq 70\%$ across varied divergence and conflict conditions by the end of the deployment and system testing phase.

SO 4.4 — Independent Recovery Operations

Achieve Hard-Level Completion Rate within Module 4 of $\geq 65\%$ across multi-step scenarios — including the integrated synthesis capstone scenario — requiring independent hard reset recovery, pushed-commit reversal, and full rebase recovery sequences without scaffolding support by the end of the acceptance testing phase.

SO 4.5 — Reduced Trial-and-Error Recovery

Reduce Average Retry Count across all Module 4 scenarios to ≤ 3 and improve Retry Transfer Accuracy to $\geq 65\%$ across eligible changed-variant retry sessions across all difficulty levels within the module by the end of the deployment and system testing phase, demonstrating that students apply systematic recovery reasoning rather than reproducing memorized command sequences.

Research Questions

RQ1 — Platform Usability and Technology Acceptance

Does GIT it!'s scenario-driven learning environment — combining a terminal-style command interface, live animated DAG visualization, and difficulty-differentiated instructional scaffolding — achieve sufficient usability and technology acceptance among third-year BSIT students at CIT-U to sustain engagement with Git practice across all four modules, as evidenced by a System Usability Scale score of ≥ 70 and TAM-based ratings of Perceived Usefulness ≥ 4.0 , Perceived Ease of Use ≥ 3.5 , and Behavioral Intention to Use ≥ 4.0 on a five-point Likert scale during the Capstone 2 evaluation period?

RQ2 — Scenario-Based Practice and Git Command Proficiency

To what extent does repeated scenario-based practice through the GIT it! Scenario Practice Workspace improve measurable Git command proficiency among third-year BSIT students at CIT-U across all four practice modules, as evidenced by a Command Accuracy Rate of $\geq 70\%$ per specific objective and a Scenario Completion Rate of $\geq 80\%$ per module across Modules 1–4 during the Capstone 2 deployment and system testing period?

RQ3 — Fading Scaffolding and Independent Repository-State Reasoning

Does the fading scaffolding model — progressively removing the expected-state diagram and contextual feedback panel as students advance from Easy to Medium to Hard difficulty — develop students' capacity to independently reason toward correct Git repository states without instructional support, as evidenced by a Hard-Level Completion Rate of $\geq 70\%$ across Modules 1–3 and $\geq 65\%$ in Module 4 during the Capstone 2 acceptance testing phase?

RQ4 — Adaptive Retry Variation and Skill Transfer

Does serving structurally changed repository variants after failed scenario sessions — rather than repeating the identical scenario — promote genuine Git skill transfer over command-sequence memorization, as evidenced by a Retry Transfer Accuracy of $\geq 65\%$ across eligible changed-variant retry sessions in Modules 3 and 4 during the deployment and system testing period?

RQ5 — Deliberate Reasoning and Reduction of Trial-and-Error Command Entry

Does the GIT it! platform's scenario design — combining a visible counted-command limit, fading instructional scaffolding, and state-based evaluation that accepts any valid path to the correct repository state — reduce trial-and-error command entry among third-year BSIT students at CIT-U in Modules 1 and 2, as evidenced by an Average Retry Count of ≤ 2 across all active Local Repository Foundations and Branching and Collaboration scenario sessions during the Capstone 2 deployment and system testing period?

SO-to-RQ Traceability

RQ	SOs Covered	KPIs	Modules
RQ1	-	SUS (≥ 70), TAM: PU ≥ 4.0 , PEOU ≥ 3.5 , BI ≥ 4.0	Platform-wide
RQ2	SO 1.1–1.6, SO 2.1–2.9, SO 3.1–3.3, SO 4.1–4.3	CAR, SCR	Modules 1–4
RQ3	SO 1.7, SO 2.10, SO 3.4, SO 4.4	HLCR	Modules 1–4
RQ4	SO 3.5, SO 4.5	RTA, ARC	Modules 3–4
RQ5	SO 1.8, SO 2.11	ARC	Modules 1–2

PART 3: Methods

System Design Overview

GIT it! is a web-based, scenario-driven learning platform delivered as a browser-accessible web application, designed to address the limited practical understanding of Git version control among third-year BSIT students at CIT-U. Rather than presenting Git commands in isolation, the platform immerses students in realistic repository problem scenarios that mirror the collaborative project conditions they encounter in courses such as Information Management 2 and Applications Development. Each scenario presents a broken or problematic repository state — such as a wrong-branch push, a merge conflict, or an accidental hard reset — and requires the student to enter Git commands that transform the repository into the expected target state.

The platform is implemented as a browser-accessible web application with a separated frontend client and backend application API. The backend manages student accounts and session data in compliance with the Philippine Data Privacy Act of 2012 (RA 10173) and applicable CIT-U institutional data privacy policies. Repository states are handled internally by the backend Repository State Simulator inside isolated server-side sessions. No external Git command-line binaries are executed, no shell commands are invoked from user input, and no connection to external Git hosting services is made at any point.

Student input passes through a command adapter controlled by the application; the adapter maps only the implemented simulator operation set to internal simulator operations. The simulator determines whether a command can be modeled by the platform, and the State-Based Evaluator separately determines whether the resulting repository state matches the scenario target. Rather than checking the specific command a student typed, the evaluator checks whether the resulting repository state matches the expected target state for that scenario step — accepting any valid command or sequence that reaches the correct final state and eliminating false negatives caused by command-string matching.

The system is governed by a No-Answer Policy: the correct command and correct command sequence are never revealed to the student at any point during or after a scenario session, at any difficulty level, on any student-facing interface surface. This policy is a binding system design constraint — absolute and non-overridable — and extends to all system components including the AI-Assisted Learning Support chatbot. Expected-state diagrams on Easy and Medium show the repository goal state but never reveal the command path needed to reach it.

The system enforces the following key design constraints:

- **No External Git Execution:** No external Git command-line binaries are invoked, no OS shell commands are executed from student input, and no connection to external Git hosting services is made at any point. All repository simulation is handled internally.
- **No-Answer Policy:** The correct command or command sequence is never revealed to the student on any interface surface at any difficulty level. This constraint is absolute and applies to the AI-Assisted Learning Support chatbot without exception.
- **RA 10173 Compliance:** Student personal data is limited to name and email address. Both are encrypted at rest and in transit in compliance with the Philippine Data Privacy Act of 2012.
- **JWT Token Security:** Access tokens have a 15-minute expiry. Refresh tokens have a 7-day expiry and are stored exclusively in an httpOnly, Secure, SameSite=Strict cookie inaccessible to JavaScript. On logout, refresh tokens are added to the Redis revocation blacklist.
- **Performance:** The system supports a minimum of 40 concurrent users during the evaluation period with architectural headroom to 100 concurrent users. The live animated DAG shall animate within 200 ms of each command response.

- **Reliability:** The system has a hard dependency on Supabase (PostgreSQL). If the database is unavailable, the system is non-operational. Upstash Redis is a soft dependency — if the cache is unavailable, the system degrades gracefully, with JWT blacklisting temporarily suspended, while remaining otherwise operational.

Development Methodology

GIT it! will be developed following the Agile software development methodology, which emphasizes iterative and incremental delivery, continuous feedback integration, and adaptive planning across short development cycles known as sprints (Dingsøyr et al., 2012). Agile is particularly well-suited to this project because the platform's core components — the Repository State Simulator, the State-Based Evaluator, the contextual feedback panel, and the difficulty-differentiated scaffolding structure — can be developed and tested independently across successive sprints before integration into a cohesive release. Each sprint produces a testable increment, allowing the team to incorporate feedback from early user testing and refine scenario content, feedback language, and interface design before final deployment.

Evaluation Approach

Primary Performance Evaluation — System-Generated KPIs

Platform effectiveness is measured through five system-generated primary KPIs derived exclusively from session and step logs, with no reliance on pre-test or post-test instruments. Scenario Completion Rate (SCR), Command Accuracy Rate (CAR), and Hard-Level Completion Rate (HLCR) primarily measure the Scenario Practice and Fading Scaffolding objectives (RQ2 and RQ3). Retry Transfer Accuracy (RTA) primarily measures the Adaptive Retry and Transfer objective (RQ4). Average Retry Count (ARC) primarily measures the Deliberate Reasoning objective (RQ5), and also serves as a convergence indicator alongside RTA for Modules 3 and 4.

Usability and Technology Acceptance — External Validation

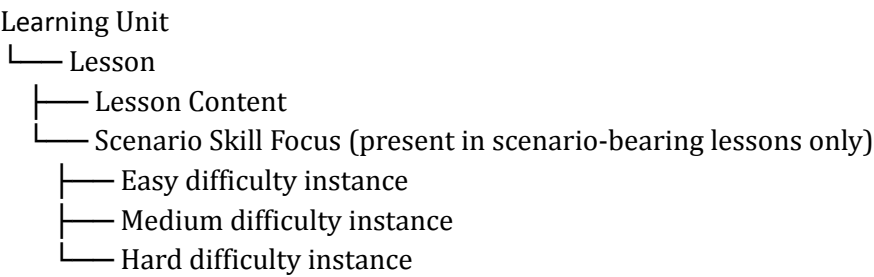
Platform usability and technology acceptance are assessed using two external validation instruments that together answer RQ1. Platform usability will be assessed using the System Usability Scale (SUS), administered after the student's first completed scenario session, with a benchmark score of ≥ 70 (Brooke, 1996; Bangor et al., 2008). Student technology acceptance will be measured through a TAM-based survey capturing Perceived Usefulness (≥ 4.0), Perceived Ease of Use (≥ 3.5), and Behavioral Intention to Use (≥ 4.0) on a five-point Likert scale (Davis, 1989; Teo, 2009). These instruments are administered during the Capstone 2 evaluation period and their results are interpreted in the context of documented Philippine HEI digital adoption barriers (Illescas et al., 2023). SUS and TAM serve as the primary measurement instruments for RQ1 and as supplementary external validation for the overall system; they do not contribute to the primary system-log KPI set (RQ2–RQ5).

PART 4: Expected System

Minimum Viable Product (MVP)

GIT it! is a web-based, scenario-driven Git learning platform designed to provide third-year BSIT students at CIT-U with a consequence-safe environment for practicing realistic version control scenarios with structured, difficulty-differentiated instructional support. The platform is organized around a curated starter content library covering foundational Git concepts through a dedicated orientation module and a set of scenario-driven lessons. The exact count, naming, ordering, and syllabus coverage of the starter library are finalized before deployment according to approved pilot needs.

The student-facing content hierarchy is structured as follows:



A **Learning Unit** is a top-level content group that organizes the Git syllabus for students in the module page.

A **Lesson** is an instructional unit within a learning unit. It contains the concept explanations, examples, command references, diagrams, and other learning materials a student needs before practice. A lesson may or may not include scenario practice — lessons without scenario skill focuses present their content directly, while lessons with scenario skill focuses attach one or more scenarios to their content.

A **Scenario Skill Focus** is a reusable Git problem pattern attached to a scenario-bearing lesson. Examples include first commit, wrong-branch commit, partial staging, detached HEAD recovery, branch cleanup, merge conflict resolution, and undo/recovery patterns. Each scenario skill focus provides three difficulty instances: Easy, Medium, and Hard.

A **Difficulty Instance** is the actual playable practice level for a scenario skill focus. Easy, Medium, and Hard target the same Git concept but differ in available scaffolding, command limits, and required independence.

A **Retry Variant** is a structurally changed version of a scenario difficulty instance used after a Failed or Abandoned session. Variants alter branch names, commit structures, file-state markers, or starting repository topology so students cannot rely on memorized command sequences.

Module 0: Orientation is the first learning unit in the platform. Its lessons contain no scenario practice. Module 0 establishes the foundational Git mental model students need before entering scenario-driven content and orients students to the platform's interface and conventions. **Orientation lesson progress is tracked by** the system, but completion of Module 0 lessons does not block scenario access — students may proceed directly to scenario practice without completing Module 0 Orientation. Module 0 does not generate scenario session logs or scenario KPI data.

Scaffolding Components

Three interface components support student reasoning within the Scenario Practice Workspace, with availability determined by difficulty level:

Live Animated DAG — Updates in real time after every simulator-processed command at all difficulty levels, rendering branch pointers, commit history, HEAD position, and parent-child commit edges. Students observe the direct consequence of each command on repository state, supporting mental model development (Sorva et al., 2013). The DAG animation completes within 200 ms of each command response.

Expected-State Diagram — A static diagram showing the target repository state the student must reach to complete the current scenario step; present on Easy and Medium levels only. Unlike the live DAG — which reflects the current, evolving state — the expected-state diagram is fixed and represents the goal. It is absent at Hard level, requiring students to infer the correct target from narrative context and the live DAG alone.

Contextual Feedback Panel — Available only on Easy-level scenarios. After a command is processed, the panel briefly summarizes the repository-state consequence of the student's action in plain language — for example, that HEAD moved to a detached commit, a branch pointer advanced, a merge conflict appeared, or the working tree became clean. It does not label commands as right or wrong, suggest the next command, or reveal the command path. Its role is to reduce beginner cognitive load by narrating the same repository transition shown visually in the live DAG, covering non-graph details such as staging-area and working-tree state that the DAG alone does not surface (Van der Kleij et al., 2015).

Difficulty-Differentiated Scaffolding Model

A central design principle of GIT it! is the progressive reduction of available scaffolding as students advance through harder difficulty levels within each scenario skill focus (Sweller et al., 2011). Scenario skill focuses are freely selectable at any time, but difficulty levels within a selected scenario follow a completion-based progression: Easy is available by default, Medium unlocks after Easy completion, and Hard unlocks after Medium completion. Each scenario offers three difficulty levels that target the same Git concept but progressively reduce the instructional support available:

Level	Scaffolding	Available Components	Cognitive Goal
Easy	Full	Live animated DAG + Expected-state diagram + Contextual feedback panel + Remaining Counted-Command Counter	Supported interpretation — student connects commands to visible repository consequences with feedback describing what changed
Medium	Partial	Live animated DAG + Expected-state diagram + Remaining Counted-Command Counter	Visually guided reasoning — student reasons toward a fixed visual target without consequence feedback
Hard	Minimal	Live animated DAG + Narrative context + Remaining Counted-Command Counter	Independent command reasoning — student resolves realistic Git problems with only the live DAG and scenario narrative

The live animated DAG remains present at all levels as the single persistent visual anchor, ensuring that students always have direct feedback on how their commands change repository state regardless of difficulty.

Command Threshold and CAR Computation

Each scenario difficulty instance includes an authored command-count policy consisting of a minimum counted-command threshold, a maximum counted-command limit, and non-counted diagnostic command patterns.

The minimum counted-command threshold represents the shortest valid counted action-command path to the target repository state for that difficulty instance and is used for Command Accuracy Rate computation. A completed primary attempt at or below this threshold is counted as command-accurate for CAR. A completed primary attempt above this threshold is still accepted as a successful completion if the target repository state is reached, but it is not counted as command-accurate for CAR.

The maximum counted-command limit represents the maximum number of counted action commands allowed in a session before the session is recorded as Failed with failure reason COUNTED_COMMAND_LIMIT_REACHED. The Remaining Counted-Command Counter is visible to the student throughout the session and decreases only for counted action commands. Simulator-processed read-only diagnostic commands that match the authored non-counted diagnostic command patterns are logged but excluded from CAR and remaining counted-command calculations. A separate submission-safety control handles repeated diagnostic, malformed, invalid, or unprocessable submissions so that inspection commands can support good Git practice without allowing session abuse.

CAR is computed from the latest completed primary attempt per student, scenario skill focus, and difficulty level during the evaluation period. Failed attempts, abandoned attempts, and Review Mode attempts are excluded from the primary CAR denominator.

Attempt Type	Included in Primary CAR?	Treatment
Failed attempt	No	Captured by SCR, ARC, failure reasons, and retry logs
Abandoned attempt	No	Captured by ARC and adaptive retry logs
Completed primary attempt above the minimum counted-command threshold	Yes	Counts as a successful completion, but not as command-accurate for CAR
Completed primary attempt at or below the minimum counted-command threshold	Yes	Counts as a successful completion and as command-accurate for CAR
Simulator-processed command matching an authored non-counted diagnostic pattern	No	Logged for audit and state review, but excluded from CAR and remaining counted-command calculations
Counted action-command total reaches the maximum counted-command limit before completion	No	Session is recorded as Failed with failure reason COUNTED_COMMAND_LIMIT_REACHED

Review Mode attempt	No	Reported separately under Review Mode completion and re-engagement indicators
---------------------	----	---

Adaptive Retry and Scenario Variation

When a student retries after a Failed or Abandoned session, the system serves a structurally different variant from the scenario's predefined pool rather than repeating the identical scenario. Variants alter branch names, commit structures, file-state markers, or starting repository topology. No Failed or Abandoned session followed by a retry results in the identical scenario being repeated when a structurally different valid variant exists. Variant IDs are logged for every attempt, enabling the system to verify that Retry Transfer Accuracy is computed only from changed-variant encounters rather than repeated identical templates.

Core Interface Components

Interface Component	Purpose
Registration and Login Screens	Allow students to create accounts and access their own progress records
Student Dashboard	Shows available learning units, active progress, completion status, retry indicators, difficulty unlocks, Command Accuracy Rate indicators, first-attempt stars, streak information, RTA-related progress, and Review access for completed difficulty instances
Unit Page	Displays the starter syllabus as student-facing learning units, beginning with Unit 1: Orientation. Each unit expands to show its lessons
Module 0: Orientation	Displays the starter syllabus as student-facing learning units, beginning with Module 0: Orientation. Each unit expands to show its lessons
Lesson Page	Presents the selected lesson's instructional content. For scenario-bearing lessons, a View Scenarios action leads the student to the scenario skill focuses attached to that lesson
Skill Focus Preview Modal	Displayed before every Start, Continue, Retry, or Review action on a Scenario Skill Focus. Shows public skill-focus information, safe demo commands, demo DAG behavior, and command purpose. Allows students to preview the scenario context before entering the Scenario Practice Workspace.
Scenario List Under Lesson	Displays the scenario skill focuses that belong to the selected lesson
Difficulty Selection / Unlocking Interface	Shows Easy, Medium, and Hard difficulty instances and enforces Easy → Medium → Hard completion-based progression within each scenario skill focus
Scenario Practice Workspace	Presents the scenario narrative, student task prompt, terminal-style command input, live animated DAG, difficulty-appropriate scaffolding components, and the Remaining Counted-Command Counter

Live Animated DAG Panel	Renders the evolving repository state after every simulator-processed command at all difficulty levels
Expected-State Diagram Panel	Shows the target repository state on Easy and Medium levels only
Contextual Feedback Panel	Summarizes the repository-state consequence of each processed command in plain language on Easy level only, without labeling commands as correct or incorrect or revealing the command path
Review Mode	Allows students to re-attempt any previously completed scenario difficulty instance with a live terminal, live DAG, and difficulty-appropriate scaffolding. Review Mode is playable, not read-only, and does not expose correct commands or post-hoc answer explanations. Not available for Failed or Abandoned sessions
Progress and KPI Indicators	Displays student-facing completion status, retry-count trends, Hard-level completion, Command Accuracy Rate, RTA-related progress, and Review activity information derived from system logs

High-Level System Workflow

Students register and log in, then access the dashboard and unit page. The unit page displays all student-facing learning units, beginning with Module 0: Orientation. Orientation lesson progress is tracked by the system, but students are not blocked from scenario access pending Module 0 completion — they may proceed directly to scenario practice at any time. Module 0 lessons build the foundational Git mental model and introduce the platform's interface conventions students need before entering scenario-driven practice.

For lessons that contain no scenario practice, the student reads the lesson content and proceeds. For scenario-bearing lessons, the student reads the lesson content, clicks View Scenarios, selects a scenario skill focus, and chooses a difficulty level. Before the Scenario Practice Workspace loads, the Skill Focus Preview Modal is presented, displaying public skill-focus information, safe demo commands, and demo DAG behavior. The student proceeds from the preview into the workspace. Easy is available first; Medium unlocks after Easy completion for the same scenario skill focus, and Hard unlocks after Medium completion.

Each scenario difficulty instance presents a professional narrative, a simulated repository state, a terminal-style command input interface, and a live animated DAG. The scaffolding available depends on difficulty level: Easy includes the live DAG, expected-state diagram, and contextual feedback panel; Medium includes the live DAG and expected-state diagram; Hard includes only the live DAG and narrative context. The Remaining Counted-Command Counter is visible at all levels.

When a command is submitted, the simulator processes it against the internal repository state and the State-Based Evaluator checks whether the resulting state satisfies the scenario's target. Any valid command or sequence reaching the correct state is accepted. When the target state is reached, the scenario difficulty instance is marked complete, the next difficulty level is unlocked when applicable, and the attempt is logged for KPI computation. Upon successful completion, the Review Mode action becomes available for that completed difficulty instance.

If a session ends without completion because the command limit is exhausted, it is marked Failed. If a student leaves or times out without explicitly completing the session, the session is marked Abandoned. Both Failed and Abandoned sessions trigger the adaptive retry logic: the system serves a structurally different variant on the next attempt when one is available. Review Mode is not available for sessions that ended as Failed or Abandoned; those outcomes lead to a retry attempt.

Review Mode is accessed from a completed scenario difficulty instance in the dashboard, unit page, or completion screen. It reuses the Scenario Practice Workspace and allows the student to attempt the completed difficulty instance again for reinforcement. Review Mode attempts are recorded separately and do not overwrite the original completion record, unlock additional levels, award first-attempt stars, update official retry counts, or alter primary KPI computation.

All session and step interactions are logged continuously for KPI computation, including started sessions, completed sessions, failed sessions, abandoned sessions, retry counts, variant IDs, command counts, failure reasons, and review-practice activity. Orientation lesson progress is tracked separately and does not generate scenario session logs or contribute to scenario KPI computation.

Content Organization Model

Content Element	Purpose
Learning Unit	Groups related lessons for the unit page; Module 0: Orientation is the first unit and contains only non-scenario lessons
Lesson	Presents instructional content; may include scenario skill focuses or may be content-only
Scenario Skill Focus	A reusable Git problem pattern attached to a scenario-bearing lesson
Difficulty Instance	Provides Easy, Medium, and Hard practice levels for a scenario skill focus
Retry Variant	Provides structurally changed scenario attempts for transfer measurement
Command-Count Policy Configuration	Stores minimum counted-command thresholds, maximum counted-command limits, and non-counted diagnostic command patterns per scenario difficulty instance

Planned Phase 2 — Not Included in Current MVP Evaluation

The current semester deliverable focuses on the student-facing scenario practice platform using a static, pre-authored starter syllabus. Administrative Management and AI-Assisted Learning Support are documented as planned Phase 2 units, excluded from current-release MVP evaluation and KPI computation.

Administrative Management consolidates all non-student platform capabilities: viewing cohort-level and student-level KPI summaries, managing student accounts and access, creating and maintaining student-facing learning units, lessons, lesson content, scenario records, scenario variants, command-count policy configurations, and reviewing administrative audit logs.

AI-Assisted Learning Support includes three planned capabilities: a conceptual Git chatbot for student-facing lesson pages, an AI-assisted lesson content drafting workflow, and an AI-assisted scenario-drafting workflow. The chatbot is subject to the No-Answer Policy without exception — it shall not reveal correct commands or command sequences for any scenario at any time. AI-generated content does not publish directly; the administrator must validate, edit, and publish all AI-generated content before students can access it.

PART 5: Discussion

Scope

GIT it! is scoped to third-year BSIT students at CIT-U enrolled in team-based development courses. The platform is organized around a static curated starter content library composed of learning units, lessons, scenario skill focuses, difficulty instances, and retry variants. Module 0: Orientation provides foundational Git and platform-orientation content through non-scenario lessons. Orientation progress is tracked by the system but does not block scenario access — students may proceed directly to scenario practice without completing Module 0. All published scenario skill focuses beyond Module 0 are freely selectable without global unit prerequisites; difficulty levels within each scenario are completion-gated according to the Easy → Medium → Hard progression. The platform is accessible as a browser-based web application on desktop, tablet, and mobile form factors. Repository states are handled internally by the backend Repository State Simulator and are never connected to live external repositories or real Git binaries. The system supports a minimum of 40 concurrent users during the evaluation period with headroom to 100. The evaluation period is bounded by a single academic semester.

Limitations

This study is bounded by several intentional scope decisions that constrain the generalizability of its findings. The platform is deployed at a single institution — CIT-U — targeting a specific population of third-year BSIT students, which means findings are not claimed to generalize to other programs or universities without further validation. The preliminary survey used to inform scenario selection was conducted with forty-four students, which, while sufficient to identify the dominant failure patterns motivating the system design, may not capture the full range of Git-related difficulties across the year level. As Illescas et al. (2023) document, Philippine-specific barriers including connectivity inequality and uneven institutional digital readiness affect technology adoption in ways that differ from Western contexts, which means care must be taken when comparing outcomes against benchmarks derived from non-Philippine populations.

The platform evaluation is conducted within a single academic semester, which may be insufficient to capture long-term retention of Git competencies. The study is designed for initial skill acquisition and consequence-safe practice rather than longitudinal mastery tracking. While the simulated repository environment reflects realistic conditions, it cannot fully replicate the complexity and unpredictability of live collaborative repositories encountered in actual industry or capstone project settings. The scenario content, lesson pages, expected-state diagrams, feedback messages, and command-count policy configurations are developed by the research team based on literature and survey data.

Module 0 orientation progress is tracked by the system, but completion is not enforced as a prerequisite for scenario access. The relationship between voluntary orientation engagement and scenario performance is treated as an observable pattern in the data rather than a causally isolated finding. Command Accuracy Rate should be interpreted as a difficulty-level command efficiency indicator rather than the sole measure of Git competence. Alternative valid command paths are accepted by the State-Based Evaluator if they reach the correct repository state, even when they exceed the authored minimum counted-command threshold. The study does not attempt to isolate the independent learning effect of individual interface components — the live DAG, the expected-state diagram, and the contextual feedback panel are evaluated as a designed system rather than as independently controlled variables. The study does not separately isolate the independent effect of dashboard visibility on student learning outcomes.

Expected Contribution

GIT it! addresses a gap that existing tools — including Learn Git Branching and GitHub Classroom — do not fill: a structured orientation module that builds foundational Git mental models before scenario practice begins, followed by consequence-safe, scenario-driven practice with targeted lesson-level concept preparation, progressively reduced instructional support, adaptive variation for authentic transfer, and a state-based evaluator that accepts any valid approach reaching the correct repository state.

Five functional units work in sequence: Module 0 orientation gives students the conceptual foundation and command vocabulary — including an explicit understanding of Git command anatomy and DAG literacy — they need before entering the terminal; the Lesson Page in each scenario-bearing lesson reinforces the specific concept relevant to the scenario at hand; the live DAG gives students real-time visibility into how commands change repository state at every difficulty level (Sorva et al., 2013); the expected-state diagram gives students a visual target to reason toward on Easy and Medium; and the contextual feedback panel on Easy translates repository consequences into plain language while the student's mental model is still forming (Van der Kleij et al., 2015). As instructional support decreases across difficulty levels, students transition from supported interpretation to independent reasoning — arriving at Hard-level scenarios equipped to resolve realistic Git problems with only the live DAG and narrative context (Sweller et al., 2011).

For students, the expected contribution is the ability to enter scenario practice conceptually prepared, reach Hard-level scenario completion, and demonstrate command execution efficiency consistent with difficulty-level mastery through Command Accuracy Rate — before those failures occur in graded projects. At the institutional level, GIT it! offers a replicable, evidence-grounded model for bridging the gap between introductory Git coverage in team-based development courses and the collaborative version control demands students face in capstone and industry-adjacent projects.

PART 6: Traceability Matrix

The matrix below maps each system feature to its RRL foundation, the specific research gap it addresses, and the research question it investigates. Every design decision in GIT it! is derived from documented evidence in the literature and targets a measurable gap not filled by existing solutions.

Identified Gap	Research Question	RRL Finding / Theme	Proposed Feature / Design Decision
No existing Git platform has been evaluated using SUS and TAM in a terminal-based, scenario-driven computing education context in a Philippine HEI setting. TAM and SUS benchmarks have not been validated for Philippine BSIT Git learners. The platform's core interaction model — terminal command input, live DAG, and fading scaffolding — introduces usability	RQ1 — Does GIT it!'s scenario-driven learning environment achieve sufficient usability and technology acceptance among third-year BSIT students at CIT-U to sustain engagement with Git practice across all four modules, as evidenced by SUS	Theme 6 — Technology Acceptance and Usability Evaluation (Davis, 1989; Teo, 2009; Brooke, 1996; Bangor et al., 2008; Orfanou et al., 2015; Illescas et al., 2023). TAM and SUS benchmarks were validated in Western organizational or teacher education contexts. Orfanou et al. (2015) confirm that $SUS \geq 70$ is an appropriately ambitious usability target for educational software. Illescas et al. (2023) document that	SUS Usability Survey: Administered to all students after their first completed scenario session. Produces a single composite usability score against the ≥ 70 threshold established for educational software (Bangor et al., 2008; Orfanou et al., 2015). TAM Student Survey: Administered after first completed module. Captures Perceived Usefulness (≥ 4.0), Perceived Ease of Use (≥ 3.5), and Behavioral Intention to Use (≥ 4.0) on a

considerations not present in existing tools (LG1 + LG2).	≥ 70 and TAM scores of $PU \geq 4.0$, $PEOU \geq 3.5$, and $BI \geq 4.0$ during the Capstone 2 evaluation period?	Philippine-specific barriers constrain direct application of Western technology acceptance benchmarks to Filipino BSIT students.	five-point Likert scale (Davis, 1989; Teo, 2009). Both instruments contextualize platform acceptance within Philippine HEI adoption constraints documented by Illescas et al. (2023). Together, these instruments serve as the primary measurement instruments for RQ1.
No curriculum-aligned, consequence-safe Git scenario practice exists for Philippine BSIT students. No existing tool evaluates student responses based on resulting repository state while accepting multiple valid command sequences. No locally contextualized platform addresses the Philippine higher education context (RG1 + LG1).	RQ2 — To what extent does repeated scenario-based practice improve measurable Git command proficiency across Modules 1–4, as evidenced by CAR $\geq 70\%$ per SO and SCR $\geq 80\%$ per module?	Theme 1 — Git Skills Gap and Industry-Education Misalignment (Garousi et al., 2019, 2020; Akdur, 2022; Illescas et al., 2023). Version control is the most consistently cited unmet competency of computing graduates globally. Graduates enter professional environments unable to manage branching strategies, resolve conflicts, or recover from destructive repository states. No locally available instructional tool addresses this gap for Philippine BSIT students.	Scenario Practice with Workspace Evaluation: Terminal-style command input, Repository State Simulator, and State-Based Evaluator accept any valid command sequence reaching the correct repository state. CAR and SCR tracked across the active scenario library. Mapped to SO 1.1–1.6, SO 2.1–2.9, SO 3.1–3.3, SO 4.1–4.3.
No Git learning platform specifically supports mental model development through real-time, command-driven DAG visualization. Competitors use static diagrams (Atlassian) or decontextualized puzzle visualization (Learn Git Branching) rather than live visualization tied to actual command execution (RG6 + SG1).	RQ3 — Does the fading scaffolding model — including the live DAG at all levels — develop students' capacity to independently reason toward correct repository states at Hard level, as evidenced by HLCR $\geq 70\%$ (Modules 1–3) and $\geq 65\%$ (Module 4)?	Theme 2 — Student Mental Model Deficits in Learning Git (Isomöttönen & Cochez, 2014; Milliken et al., 2021; Embury, 2019; Wagner & Thurner, 2024). Novice developers lack a clear mental model of repository state transitions. Students across all skill levels routinely fail to understand how their commands change the repository. No existing Git learning tool addresses mental model formation through interactive visualization tied to live command execution.	Live Animated DAG Panel: Updates within 200 ms of every simulator-processed command at all difficulty levels. Renders branch pointers, commit history, HEAD position, and parent-child edges in real time so students observe repository state change as a direct consequence of each command (Sorva et al., 2013). Present at Easy, Medium, and Hard — the single persistent visual anchor across all scaffolding tiers. Mapped to SO 1.7, SO 2.10, SO 3.4, SO 4.4.
No existing Git learning platform provides a structured orientation unit covering repository structure, commit history, HEAD, DAG	RQ2 — To what extent does repeated scenario-based practice improve measurable Git	Theme 2 — Student Mental Model Deficits (continued) and Theme 4 — Cognitive Load Management and Instructional Scaffolding	Module 0: Orientation: Six sequenced non-scenario lessons covering Git mental model, four-area diagram, commit history, DAG literacy, command anatomy,

literacy, and command anatomy before students enter scenario-driven practice (SG1 + RG3 + LG1).	command proficiency, as evidenced by CAR $\geq 70\%$ per SO and SCR $\geq 80\%$ per module?	(Sweller, 1988; Sweller et al., 2011; Mayer, 2009; Ling & Chiang, 2022). Students who lack a foundational mental model before scenario practice encounter maximal extraneous cognitive load. No competitor provides a structured orientation unit that builds foundational Git mental models before students enter interactive practice.	diagnostic command fluency (git status, git log, git diff), reflog reading, branch divergence reading, and platform conventions. Orientation progress is tracked by the system; completion is encouraged but not enforced as a prerequisite for scenario access. Mapped to SO 1.1–SO 4.3 (foundational preparation).
No competitor contextualizes Git commands within professional scenario narratives. GitHub Skills uses generic module titles; Learn Git Branching uses decontextualized puzzle framing; Atlassian is reference-only with no scenario context. No empirically validated, scenario-based Git learning platform exists in the literature (RG1 + SG1).	RQ2 — To what extent does repeated scenario-based practice improve measurable Git command proficiency, as evidenced by CAR $\geq 70\%$ per SO and SCR $\geq 80\%$ per module?	Theme 3 — Simulation-Based and Scenario-Based Learning (Chernikova et al., 2020; Terzopoulos & Satratzemi, 2023; Mio et al., 2019; López Belmonte et al., 2024). Simulation-based learning significantly improves outcomes in higher education when functional fidelity is high. Learners are more engaged when technical tasks are framed within realistic professional narratives. Application-level skills show weaker improvement (54%) without scenario-based practice that bridges conceptual understanding and actual execution. The CS domain is underrepresented in SBL research.	Professional Scenario Narrative Framing: Every scenario is presented as a realistic software project episode. Narrative context activates prior knowledge and frames each practice session as a realistic professional challenge (Terzopoulos & Satratzemi, 2023). Per-Scenario Lesson Page: Each scenario-bearing lesson page introduces the Git concepts and commands relevant to that scenario, accessible before and during practice. Skill Focus Preview Modal: Displayed before every Start, Continue, Retry, or Review action, showing safe demo commands, demo DAG behavior, and command purpose to orient students before entering the workspace. Mapped to SO 1.1–1.6, SO 2.1–2.9, SO 3.1–3.3, SO 4.1–4.3.
No Git learning tool implements graduated, difficulty-differentiated instructional support with systematic scaffold removal across difficulty levels. GitHub Skills offers no scaffolding variation. Learn Git Branching provides uniform hint availability across all levels. No competitor measures whether	RQ3 — Does the fading scaffolding model develop students' capacity to independently reason toward correct repository states, as evidenced by HLCR $\geq 70\%$ (Modules 1–3) and $\geq 65\%$ (Module 4)?	Theme 4 — Cognitive Load Management and Instructional Scaffolding (Sweller et al., 2011; Van der Kleij et al., 2015; Ling & Chiang, 2022). Fading scaffolding significantly improves long-term skill retention compared to constant support. Providing full support at all levels prevents the development of independent performance. No competitor implements a	Difficulty-Differentiated Fading Scaffolding Model: Easy (full support: live DAG + expected-state diagram + contextual feedback panel + command counter); Medium (partial: live DAG + expected-state diagram + command counter); Hard (minimal: live DAG + narrative + command counter only). Completion-Based Difficulty Progression:

scaffold reduction produces independent Hard-level performance (RG3 + SG1).		graduated, difficulty-differentiated scaffolding model structurally embedded into scenario design.	Easy available by default; Medium unlocks after Easy completion; Hard unlocks after Medium completion. Expected-State Diagram Panel: Present on Easy and Medium only. Mapped to SO 1.7, SO 2.10, SO 3.4, SO 4.4.
No competitor provides elaborated, consequence-narrating feedback tied to repository state. GitHub Skills delivers pass/fail only. Learn Git Branching offers objective-level hints but no error-diagnostic feedback. Atlassian has no feedback mechanism. No existing tool narrates the repository-state consequence of each command without revealing the correct answer (RG2 + SG2).	RQ2 — To what extent does repeated scenario-based practice improve measurable Git command proficiency, as evidenced by CAR $\geq 70\%$ per SO? RQ5 — Does the platform's scenario design reduce trial-and-error command entry, as evidenced by ARC ≤ 2 in Modules 1–2?	Theme 4 — Cognitive Load Management (continued) and Theme 5 — Automated Feedback Design (Van der Kleij et al., 2015; Keuning et al., 2019; Hao et al., 2022). Elaborated feedback produces learning effect sizes ten times larger than simple correctness indicators. Insufficiently detailed feedback drives gaming behavior — repeated blind submission without engaging with the underlying concept. This is especially destructive in sequential command-learning contexts like Git where misordered operations compound errors.	Contextual Feedback Panel (Easy Level Only): After each command, narrates the repository-state consequence in plain language without labeling commands as correct or incorrect, suggesting the next command, or revealing the correct sequence (Van der Kleij et al., 2015). Counted-Command Limit Enforcement: Visible Remaining Counted-Command Counter decrements with each action command. Diagnostic commands are non-counted and freely available at all difficulty levels. Mapped to SO 1.8, SO 2.11, SO 1.1–1.6, SO 2.1–2.9.
No competitor prevents command memorization by serving structurally changed scenario variants after failed or abandoned sessions. All three analyzed competitors either repeat the same exercise or offer no retry mechanism at all. Keuning et al. (2019) identify a structural gap in next-step guidance and transfer support that no existing tool fills (RG2 + SG2).	RQ4 — Does serving structurally changed repository variants after failed scenario sessions promote genuine Git skill transfer, as evidenced by RTA $\geq 65\%$ across eligible changed-variant retry sessions in Modules 3–4? RQ5 — Does the platform's scenario design reduce trial-and-error command entry, as evidenced by ARC ≤ 2 in Modules 1–2?	Theme 5 — Automated Feedback and Authentic Transfer (Van der Kleij et al., 2015; Keuning et al., 2019; Hao et al., 2022). Students who receive identical scenarios on retry reproduce memorized command sequences rather than applying underlying concepts. Variant-based retry forces conceptual understanding over procedural recall. No study has applied adaptive variant-based retry specifically to Git scenario learning.	Adaptive Retry with Structurally Changed Variants: When a session ends as Failed or Abandoned, the system serves a structurally different scenario variant on the next attempt. Variant IDs are logged for every attempt. Retry Transfer Accuracy (RTA): Computed exclusively from eligible changed-variant retry sessions. Review Mode: Allows re-practice of completed difficulty instances without affecting primary KPI computation. Not available for Failed or Abandoned sessions. Mapped to SO 3.5, SO 4.5.

PART 7: References

- Akdur, D. (2022). Analysis of the software engineering skills gap in the industry. *ACM Transactions on Computing Education*, 23(2), 1–25. <https://doi.org/10.1145/3567837>
- Bangor, A., Kortum, P., & Miller, J. (2008). An empirical evaluation of the system usability scale. *International Journal of Human-Computer Interaction*, 24(6), 574–594. <https://doi.org/10.1080/10447310802205776>
- Beck, K., et al. (2001). Manifesto for agile software development. Agile Alliance. <https://agilemanifesto.org>
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for agile software development*. Agile Alliance. <https://agilemanifesto.org>
- Beckman, M., Çetinkaya-Rundel, M., & Horton, N. J. (2020). Integrating version control into the data science curriculum. *Journal of Statistics and Data Science Education*, 28(2), 145–156.
- Brooke, J. (1986). SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & A. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.
- Chernikova, O., Heitzmann, N., Stadler, M., Holzberger, D., Seidel, T., & Fischer, F. (2020). Simulation-based learning in higher education: A meta-analysis. *Review of Educational Research*, 90(4), 499–541. <https://doi.org/10.3102/0034654320933544>
- Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, 13(3), 319–340. <https://doi.org/10.2307/249008>
- Deci, E. L., & Ryan, R. M. (2000). Self-determination theory and the facilitation of intrinsic motivation. *American Psychologist*, 55(1), 68–78.
- Deci, E. L., & Ryan, R. M. (2000). The "what" and "why" of goal pursuits: Human needs and the self-determination of behavior. *Psychological Inquiry*, 11(4), 227–268.
- Dingsøyr, T., Nerur, S., Balijepally, V., & Moe, N. B. (2012). A decade of agile methodologies: Towards explaining agile software development. *Journal of Systems and Software*, 85(6), 1213–1221.
- Embury, S. (2019). Teaching version control to undergraduate software engineers. *Journal of Computing Sciences in Colleges*, 34(6), 121–128.
- Garousi, V., Giray, G., Tüzün, E., Catal, C., & Felderer, M. (2019). Aligning software engineering education with industrial needs: A meta-analysis. *IEEE Access*, 7, 117527–117543. <https://doi.org/10.1109/ACCESS.2019.2936660>
- Garousi, V., Giray, G., Tuzun, E., Catal, C., & Felderer, M. (2020). Closing the gap between software engineering education and industrial needs. *IEEE Software*, 37(2), 68–77.
- Gitinabard, N., Okoilu, R., Xu, Y., Heckman, S., Barnes, T., & Lynch, C. (2020). Student teamwork on programming projects: What can GitHub logs show us? In *Proceedings of*

- the 51st ACM Technical Symposium on Computer Science Education (SIGCSE)*.
<https://doi.org/10.1145/3328778.3366950>
- Guerrero-Higueras, Á. M., Llamas, M., & González, M. Á. (2020). Academic integrity in version control systems: A study of student practices in Git. *IEEE Access*, 8, 125430–125440.
- Hao, Q., Barnes, B., Wright, E., & Branch, R. M. (2022). The effects of worked examples on computer programming problem-solving. *Educational Technology Research and Development*, 70(3), 895–918.
- Isomöttönen, V., & Cochez, M. (2014). Challenges of teaching Git to CS students. In *Proceedings of the 14th Koli Calling International Conference on Computing Education Research* (pp. 81–90). <https://doi.org/10.1145/2674683.2674694>
- Jabrayilzade, F., Isomöttönen, V., & Cochez, M. (2022). Students’ self-assessed Git skills in software engineering courses. *Journal of Systems and Software*, 183, 111114. <https://doi.org/10.1016/j.jss.2021.111114>
- Keuning, H., Jeuring, J., & Heeren, B. (2019). A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education*, 19(1), 1–43.
- Ling, H. C., & Chiang, C. H. (2022). Effects of scaffolding on programming learning outcomes: A meta-analysis. *Journal of Educational Computing Research*, 60(4), 867–894.
- López Belmonte, J., Moreno-Guerrero, A. J., & Marín-Marín, J. A. (2024). Scenario-based learning in higher education: A bibliometric analysis. *Interactive Learning Environments*, 32(1), 112–128.
- Mio, C., Cestnik, B., & Karpati, A. (2019). Simulation-based learning in computer science: A systematic review. In *Proceedings of the 21st International Conference on Interactive Collaborative Learning (ICL 2018)* (pp. 123–135). Springer.
- Mayer, R. E. (2009). *Multimedia learning* (2nd ed.). Cambridge University Press.
- Milliken, G., Nguyen, S., & Steeves, V. (2021). A behavioral approach to understanding the Git experience. In *Proceedings of the Hawaii International Conference on System Sciences*.
- Orfanou, K., Tselios, N., & Katsanos, C. (2015). Perceived usability evaluation of learning management systems: Empirical evaluation of the System Usability Scale. *The International Review of Research in Open and Distributed Learning*, 16(2), 227–246.
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner’s approach* (9th ed.). McGraw-Hill Education.
- Sorva, J., Karavirta, V., & Malmi, L. (2013). A review of generic program visualization systems for introductory programming education. *ACM Transactions on Computing Education*, 13(4), 1–64.
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285. https://doi.org/10.1207/s15516709cog1202_4
- Sweller, J., Ayres, P., & Kalyuga, S. (2011). *Cognitive load theory*. Springer. <https://doi.org/10.1007/978-1-4419-8126-4>

- Terzopoulos, G., & Satratzemi, M. (2023). Scenario-based learning in computer science education: A systematic literature review. *Education and Information Technologies*.
- Van der Kleij, F. M., Feskens, R. C. W., & Eggen, T. J. H. M. (2015). Effects of feedback in a computer-based learning environment on students' learning outcomes: A meta-analysis. *Review of Educational Research*, 85(4), 475–511. <https://doi.org/10.3102/0034654314564881>
- Wagner, G., & Thurner, L. (2025). Teaching tip: Rethinking how we teach Git. *Journal of Information Systems Education*, 36(1). <https://aisel.aisnet.org/jise/vol36/iss1/1/>
- Zagalsky, A., Feliciano, J., Storey, M.-A., Zhao, Y., & Wang, W. (2015). The emergence of GitHub as a collaborative platform for education. In *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing* (pp. 1906–1917). <https://doi.org/10.1145/2675133.2675284>